EE 475 HW1



AHMET BURAK EKİN

2021401141

Question 1

a) For this process Min-max normalization is applied.

The function scales pixel intensities linearly from the dynamic range $[\min(A), \max(A)]$ to $[0,1]$ using dynamic range stretching (linear scaling):

$$f(x,y) = \frac{A(x,y) - \min(A)}{\max(A) - \min(A)}$$

*(*Assuming $\max(A) \neq \min(A)$*).*

The required operations are:

1. Computing the global minimum and maximum intensity values (min and max operations).
2. Level-shifting (subtracting $\min(A)$).
3. Normalizing by the dynamic range (dividing by $\max(A) - \min(A)$).

b) Python code:

```
scaled_A = (A.astype(float) - A.min()) / (A.max() - A.min())
```

Question 2

a)

```
75-image/hw1/hw1.py
Dosya: Chronometer.tif
Resolution: 2140 x 2136 pixels
Toplam Piksel: 4,571,040

Dosya: Chronometerlow2.tif
Resolution: 1070 x 1068 pixels
Toplam Piksel: 1,142,760

Dosya: Chronometerlow5.tif
Resolution: 428 x 428 pixels
Toplam Piksel: 183,184

Dosya: Chronometerlow34.tif
Resolution: 535 x 712 pixels
Toplam Piksel: 380,920
```

Figure1

b) Code

c) After implementing bilinear and bicubic interpolation, bicubic outputs look visibly clearer. In particular, the white parts and details look sharper. When comparing the results of reduced image 2(Figure3)and reduced image 34(Figure4), it is hard to see any difference with the naked eye. However, reduced image 5(Figure5) clearly gives a worse result.

Since visual inspection is not enough, I calculated the MSE(Mean Squared Error) and PSNR(Peak Signal-to-Noise Ratio) values. As shown in Figure 2, bicubic interpolation achieves lower MSE and higher PSNR values in all cases. Also, reduced image 5 has the highest MSE, showing it is the furthest from the original image.

```
--- PERFORMANS METRİKLERİ (c ŞIKKI) ---
Dosya                    | Yöntem     | MSE        | PSNR (dB)
-------------------------------------------------------------
Chronometerlow2.tif      | Bilinear   | 15.08      | 36.35
                         | Bicubic    | 12.89      | 37.03
-------------------------------------------------------------
Chronometerlow5.tif      | Bilinear   | 242.67     | 24.28
                         | Bicubic    | 250.94     | 24.14
-------------------------------------------------------------
Chronometerlow34.tif     | Bilinear   | 48.16      | 31.30
                         | Bicubic    | 35.67      | 32.61
```

Figure 2

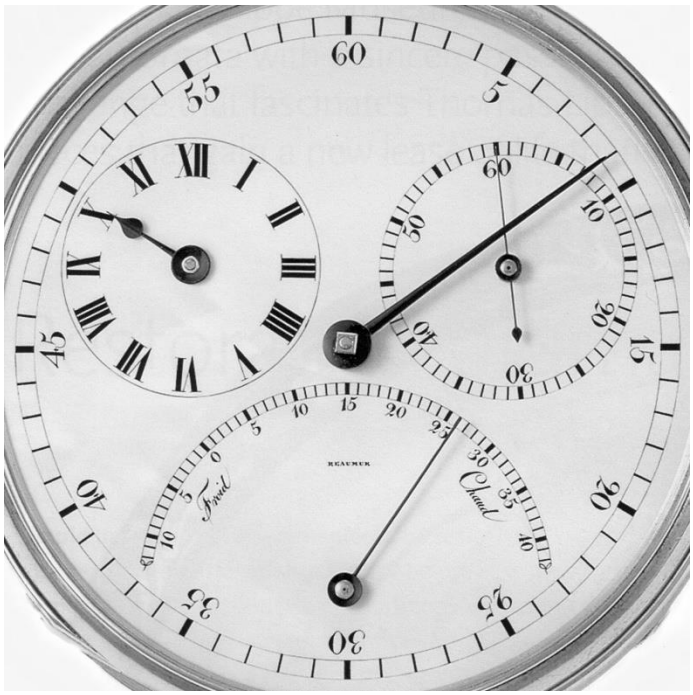


Figure 3

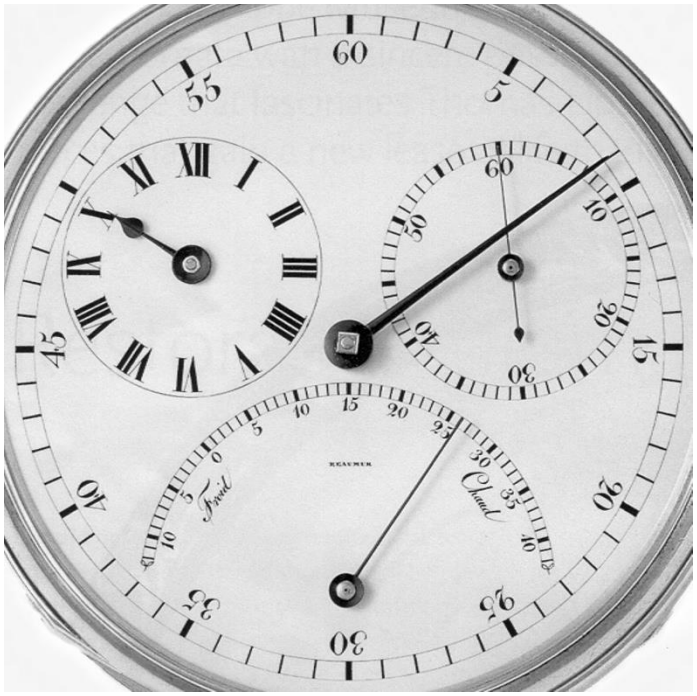


Figure 4

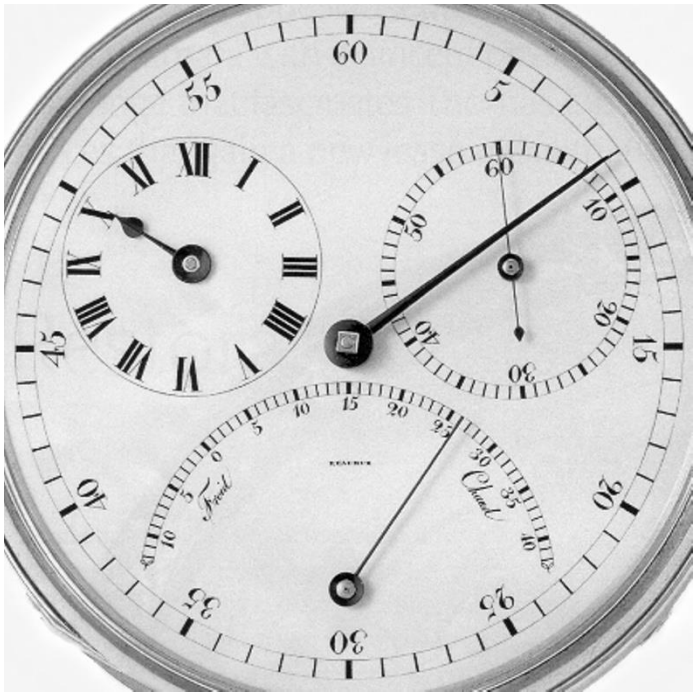


Figure 5

APPENDIX SOURCE CODE

```
import numpy as np
import math
from pathlib import Path
from PIL import Image

# b şıkkı için interpolasyon algoritmaları
def simple_bilinear_interpolation(img, target_h, target_w):
    old_h, old_w = img.shape
    out = np.zeros((target_h, target_w), dtype=np.float32)

    for i in range(target_h):
        for j in range(target_w):
            y = i * (old_h / target_h)
            x = j * (old_w / target_w)

            y0 = int(y)
            x0 = int(x)
            y1 = min(y0 + 1, old_h - 1)
            x1 = min(x0 + 1, old_w - 1)

            dy = y - y0
            dx = x - x0

            top_left = img[y0, x0]
            top_right = img[y0, x1]
            bottom_left = img[y1, x0]
            bottom_right = img[y1, x1]

            out[i, j] = (
                top_left * (1 - dx) * (1 - dy) +
                top_right * dx * (1 - dy) +
                bottom_left * (1 - dx) * dy +
                bottom_right * dx * dy
            )

    return np.clip(out, 0, 255).astype(np.uint8)

# bicubic için ağırlık fonksiyonu
def cubic_weight(x):
    x = abs(x)
    if x <= 1:
        return 1.5 * (x**3) - 2.5 * (x**2) + 1
    elif x < 2:
        return -0.5 * (x**3) + 2.5 * (x**2) - 4 * x + 2
    return 0

def simple_bicubic_interpolation(img, target_h, target_w):
    old_h, old_w = img.shape
    out = np.zeros((target_h, target_w), dtype=np.float32)

    # kenarlarda patlamaması için padding ekliyoruz
    img_pad = np.pad(img, pad_width=2, mode='edge').astype(np.float32)
```

```
    for i in range(target_h):
        for j in range(target_w):
            y = i * (old_h / target_h)
            x = j * (old_w / target_w)

            y_int = int(y)
            x_int = int(x)

            dy = y - y_int
            dx = x - x_int

            pixel_val = 0.0

            # 4x4 komşuluk
            for m in range(-1, 3):
                for n in range(-1, 3):
                    val = img_pad[y_int + m + 2, x_int + n + 2]
                    weight_y = cubic_weight(m - dy)
                    weight_x = cubic_weight(n - dx)
                    pixel_val += val * weight_y * weight_x

            out[i, j] = pixel_val

    return np.clip(out, 0, 255).astype(np.uint8)

# klasör yolu
folder_path = Path(r"C:\Users\abee\Desktop\5-1\475-image\hw1")

# a şıkkı: dosya analizi
print(f"{'Dosya Adı':<25} | {'Çözünürlük (GxY)':<18} | {'Toplam Piksel'}")
print("-" * 60)

for file in sorted(folder_path.glob("*.tif")):
    with Image.open(file) as img:
        w, h = img.size
        print(f"{file.name:<25} | {f'{w} x {h}':<18} | {w * h:,}")
print("\n")

# b şıkkı: interpolasyon implementasyonu
orig_file_path = folder_path / "Chronometer.tif"

# orijinal görüntüyü referans olarak al
with Image.open(orig_file_path) as img_orig:
    original_img = np.array(img_orig.convert('L'))

target_height, target_width = original_img.shape

# işlenecek düşük çözünürlüklü dosyalar
reduced_files = ["Chronometerlow2.tif", "Chronometerlow5.tif", "Chronometerlow34.tif"]

for file_name in reduced_files:
    reduced_file_path = folder_path / file_name
    print(f"[{file_name}] işleniyor...")

    with Image.open(reduced_file_path) as img_sub:
        reduced_img = np.array(img_sub.convert('L'))
```

```
    # bilinear
    print("  -> Bilinear hesaplanıyor (biraz sürebilir)")
    bilinear_result = simple_bilinear_interpolation(reduced_img, target_height,
target_width)

    # bicubic
    print("  -> Bicubic hesaplanıyor (daha uzun sürer)")
    bicubic_result = simple_bicubic_interpolation(reduced_img, target_height, target_width)

    # dosyaların üzerine yazmaması için isimleri dinamik oluştur
    out_bilinear_name = f"result_bilinear_{file_name}"
    out_bicubic_name = f"result_bicubic_{file_name}"

    Image.fromarray(bilinear_result).save(folder_path / out_bilinear_name)
    Image.fromarray(bicubic_result).save(folder_path / out_bicubic_name)

    print(f"  -> Kaydedildi: {out_bilinear_name} ve {out_bicubic_name}\n")

print("Tüm işlemler bitti.")

#C ŞIKKI

def calculate_metrics(original, reconstructed):
    # İki matris arasındaki karesel farkların ortalaması
    mse = np.mean((original.astype(float) - reconstructed.astype(float)) ** 2)

    if mse == 0:  # Görüntüler birebir aynıysa
        return 0, float('inf')

    max_pixel = 255.0
    psnr = 10 * math.log10((max_pixel ** 2) / mse)

    return mse, psnr

print("\n--- PERFORMANS METRİKLERİ (c ŞIKKI) ---")
print(f"{'Dosya':<25} | {'Yöntem':<10} | {'MSE':<10} | {'PSNR (dB)'}")
print("-" * 65)

for file_name in reduced_files:
    # Kaydedilen görüntüleri tekrar oku
    bilinear_img = np.array(Image.open(folder_path / f"result_bilinear_{file_name}"))
    bicubic_img = np.array(Image.open(folder_path / f"result_bicubic_{file_name}"))

    # Metrikleri hesapla
    mse_bil, psnr_bil = calculate_metrics(original_img, bilinear_img)
    mse_bic, psnr_bic = calculate_metrics(original_img, bicubic_img)

    print(f"{file_name:<25} | Bilinear   | {mse_bil:<10.2f} | {psnr_bil:.2f}")
    print(f"{'':<25} | Bicubic    | {mse_bic:<10.2f} | {psnr_bic:.2f}")
    print("-" * 65)
```